

1. [江科大笔记](#)

2. mcu上电启动流程

MCU的上电启动流程通常包括**上电复位**，接着执行**启动程序 (Bootloader)**，该程序会初始化**系统时钟和外设**，最后跳转到**用户主程序**开始执行。

\1. 上电复位 (Power-on Reset)

- **上电**：MCU接通电源后，内部复位电路会执行复位操作，将所有寄存器、状态标志和内存初始化为预设的默认值。
- **复位向量**：复位后，MCU会根据配置（如BOOT引脚电平）或默认设置，从一个固定的地址（通常是Flash或ROM的起始地址）开始执行第一条指令，这个地址称为复位向量。

\2. 启动程序 (Bootloader)

- **硬件初始化**：在复位向量处，MCU开始执行Bootloader程序，它首先会执行关键的硬件初始化，包括设置系统时钟、时钟分频器等，确保系统能以正确的速度运行。
- **内存和外设初始化**：Bootloader会继续初始化其他硬件资源，如内存控制器、各种外设接口（如串行接口、定时器等）和中断向量表。
- **系统自检**：在某些MCU中，Bootloader可能会进行一些系统自检，以确保硬件正常工作。

\3. 用户主程序执行

- **主程序加载**：Bootloader加载用户编写的主应用程序（应用程序）到内存中。
- **跳转到main函数**：完成必要的初始化后，Bootloader会将控制权交给主程序，通常是通过跳转到C语言的 `main()` 函数来启动用户程序执行。

3. c语言编译流程

1. 预处理：展开宏和头文件，生成 `.i` 文件；
2. 编译：将预处理文件转汇编代码，生成 `.s` 文件；
3. 汇编：将汇编代码转机器码，生成 `.o` 目标文件；
4. 链接：合并目标文件和库，通过链接脚本分配地址，生成 `.elf` 可执行文件；嵌入式中还需将 `.elf` 转换为 `bin/hex` 文件用于烧录，核心是通过链接脚本匹配硬件内存布局。

4. mos管和三极管

MOS管是电压控制的元件，而三极管是电流控制的元件。

三极管比MOS管的功耗大。

三极管比较便宜适合做一些小电流电路的开关

三极管耐高压大电流

5. 堆和栈，队列

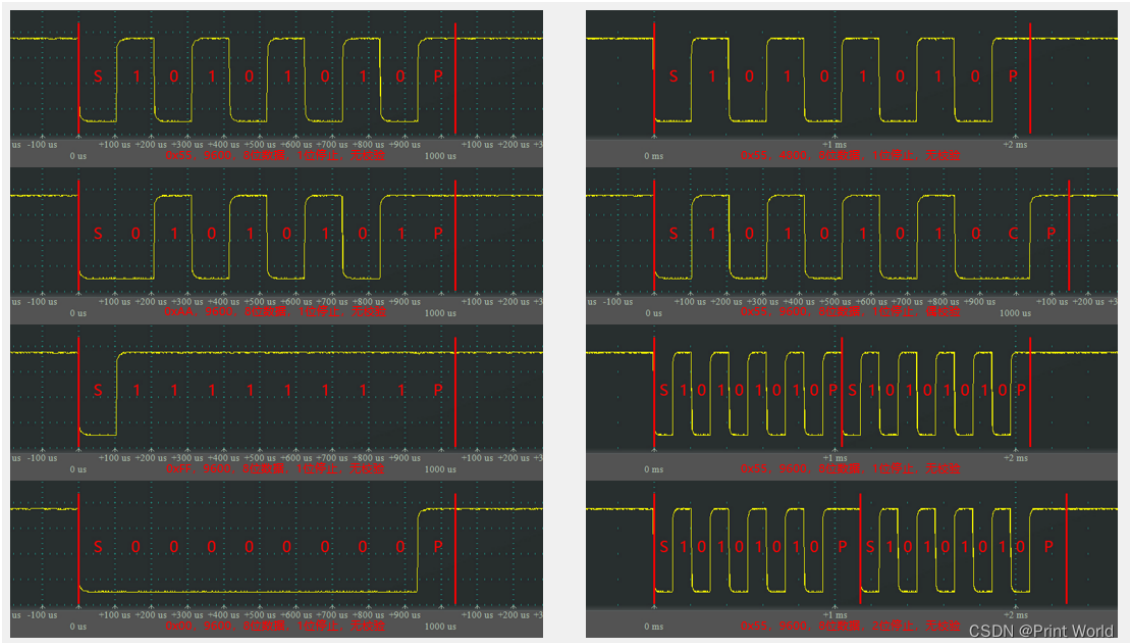
栈是后进先出 (LIFO) 的数据结构，常用于函数调用和局部变量，内存由系统自动管理。***队列*** 是**先进先出 (FIFO)** 的数据结构，用于任务调度等场景。**堆**（数据结构）是一种特殊的树形结构，有最大堆和最小堆之分，常用于堆排序。**堆**（内存）是程序运行时动态分配的内存区域，需要手动管理分配和释放。

6. 野指针，内存泄漏

野指针是悬空指针，指向一个不再合法（如已释放）的内存地址，操作它会引发程序崩溃或数据损坏。内存泄漏是指程序动态分配了内存，但用完后未释放，导致这部分内存无法回收，长期累积会耗尽系统内存。野指针可能导致内存泄漏，但内存泄漏并不一定由野指针引起，还可能因忘记释放内存或丢失指针引用而发生。

7. 什么是UART通信？它的时序是怎样的？

UART（通用异步收发器）是一种全双工的串行通信协议，它通过单根发送线（TX）和接收线（RX）传输数据，双方必须事先约定好传输速率（波特率）和数据格式。其时序基于**数据帧格式**，在没有时钟信号的情况下实现同步，具体时序结构为：**起始位 + 数据位（5-9位） + 可选的奇偶校验位 + 停止位**。



8. 什么是spi通信？它的时序是怎样的？

SPI（串行外设接口）是一种高速、全双工的同步串行通信协议，通常用于微控制器和外设之间的数据传输，它依赖于四根线（MISO、MOSI、SCLK、CS）进行通信。其时序由主机产生的串行时钟（SCK）控制，在每次时钟边沿（上升沿或下降沿）进行数据采样和输出，具体取决于时钟极性（CPOL）和时钟相位（CPHA）的配置

9. I2C协议中的总线仲裁是如何工作的？如何在STM32中实现多从设备的I2C通信？

I2C是一种串行通信协议，使用两根线（SCL串行时钟线和SDA串行数据线）进行主从通信，其时序主要包括起始信号、数据传输（包括应答）和停止信号。起始信号由主机在SCL为高电平期间，将SDA从高变低产生。数据传输时，主机在SCL低电平期间发送数据，从机在SCL高电平期间读取数据，并在传输完一字节后通过SDA的应答信号（ACK/NACK）确认。停止信号是主机在SCL为高电平期间，将SDA从低变高产生

10. TCP和UDP的区别是什么？

11. 在嵌入式系统中什么时候选择UDP？

在嵌入式系统中，当对实时性要求高但对数据可靠性要求较低时，应选择UDP。这包括诸如音频/视频流、在线游戏、DNS查询、以及需要快速响应的传感器数据采集等场景

12. SPI和I2C的优缺点是什么？什么时候更适合使用哪一个？

SPI 适用于高速、大量数据传输的场景，而 I2C 适用于低速、连接多个设备并希望节省引脚资源的场景。SPI 的主要优点是速度快（全双工），而 I2C 的主要优点是只需要两根线（SDA 和 SCL）即可连接多个设备，支持多主节点。

13. 在嵌入式系统中，如何优化TCP/IP通信的效率？

14. PWM信号如何用于控制伺服电机？其时序如何实现精确控制？

15.

PWM（脉冲宽度调制）信号通过改变其占空比来控制伺服电机的角度。精确控制时序是通过固定脉冲周期的同时，调整脉冲的高电平持续时间（即占空比）来实现的。

16. 在嵌入式系统中，volatile关键字的作用是什么？

在嵌入式系统中，`volatile` 关键字的作用是**防止编译器对变量进行优化**。它告诉编译器，变量的值可能在程序控制之外被改变，因此每次访问该变量时都必须**直接从内存中读取**，而不是使用保存在寄存器中的缓存值。

17. C语言中的指针与数组有什么区别？如何使用它们进行内存管理？

C语言中的数组是一块连续的内存空间，用于存储同类型的数据，而指针是一个变量，它存储的是另一个变量的内存地址。两者在内存管理中的主要区别是，数组在声明时就分配了固定的内存空间，而指针的内存空间大小取决于它存储的地址类型，并且在内存管理时，需要手动通过 `malloc` 等函数分配和释放指针指向的内存空间。指针的大小为4字节

18. 如何在C语言中实现位操作？

在 C 语言中，可以使用六种位运算符来实现位操作：**按位与** `&`、**按位或** `|`、**按位取反** `~`、**按位异或** `^`，以及**左移** `<<` 和**右移** `>>`。

19. 什么是函数指针？在嵌入式开发中如何应用？

函数指针是指向函数入口地址的指针，可以在嵌入式开发中实现灵活的函数调用、事件处理和代码复用。在嵌入式开发中，它常用于实现回调函数、状态机和中断服务子程序（ISR）的注册。

20. C语言中的结构体对齐（struct padding）是什么？为什么要注意它？

C语言中的结构体对齐（struct padding）是指在编译时，为了提高内存访问效率和满足硬件要求，在结构体成员之间或末尾自动填充额外字节（padding）的过程。主要原因是**提高访问效率**，因为许多处理器以固定大小的块来读取内存，对齐的成员可以一次性读取；其次是**平台兼容性**，一些硬件平台不允许或会因访问未对齐内存而报错，因此对齐是必要的。

21. 如何实现C语言中的中断处理函数？

关键步骤是编写一个中断服务程序（ISR），并在中断向量表中注册该程序，以便在硬件触发中断时，系统能够将控制权转移到您的 ISR 来执行相应的处理代码。

22. 在C中如何处理大数据量的内存管理？

1. **静态分配优先**：固定数据用全局 / 静态数组，无碎片、高稳定；
2. **动态分配优化**：用内存池替代裸 `malloc`，减少碎片，提升速度；
3. **硬件层扩展**：结合外部 Flash/SDRAM/SD 卡突破内部 RAM 限制；

23. C语言中的死循环如何影响嵌入式系统？如何避免它？

C语言中的死循环会耗尽嵌入式系统的CPU资源，导致系统无响应、死机，并且无法执行其他任务，因为它会阻止程序进入循环后的任何代码。为了避免这种情况，应该使用**条件判断来跳出循环**，比如在循环内部设置一个**条件**，当这个条件满足时，就使用 `break` 关键字来退出循环，或者使用其他逻辑来终止循环。

24. 如何优化C语言代码以减少嵌入式系统中的内存占用？

根据变量的实际需求选择最小的数据类型，例如使用 `uint8_t` 代替 `int` 来存储较小的整数。避免不必要的全局变量和静态变量，优先使用栈上的局部变量，因为它们在超出作用域后会自动释放。动态分配内存时，确保在使用完毕后立即释放，以防止内存泄露。将频繁调用的函数进行内联优化，以减少函数调用开销。

25. 解释C语言中的堆和栈的区别以及在嵌入式系统中的使用场景。

C语言的栈是自动分配和释放的，用于存储函数局部变量和参数，速度快但大小有限；堆是程序员手动分配和释放的，用于动态内存，灵活性高但易出错且速度慢。

26. 在STM32中，如何配置一个GPIO为中断模式？

在STM32中配置GPIO为中断模式，需要按照以下步骤进行：1. 打开时钟；2. 配置GPIO为输入模式；3. 将GPIO连接到外部中断线；4. 配置中断触发方式和响应方式；5. 使能中断并配置NVIC优先级。

27. 如何使用STM32的DMA控制器来实现高效数据传输？

要使用STM32的DMA控制器实现高效数据传输，首先需要进行初始化配置，设置源地址、目标地址、传输数据量、传输方向和优先级等参数。然后，通过外设请求或软件触发来启动DMA传输，DMA控制器会接管总线，在CPU不干预的情况下自动完成数据搬运。最后，在DMA传输完成后，可以根据配置触发中断或事件，以便CPU执行后续操作。

28. 如何在STM32中使用ADC采集模拟信号？

要在STM32中使用ADC采集模拟信号，首先需要使用STM32CubeMX配置ADC外设（如选择通道、分辨率、采样时间）并生成初始化代码。然后，在main.c文件中，通过HAL库函数如[HAL_ADC_Start](#)启动ADC转换，使用[HAL_ADC_PollForConversion](#)等待转换完成，并通过[HAL_ADC_GetValue](#)获取转换结果。

29. 如何配置STM32的PWM输出以控制LED的亮度？

要配置STM32的PWM输出以控制LED亮度，您需要使用STM32CubeMX或代码手动配置一个定时器（TIM）的外设。主要步骤包括：选择一个定时器通道，设置定时器的预分频器和计数周期来确定PWM频率，然后将该通道设置为PWM生成模式，并通过修改占空比来调节LED亮度。

30. 在STM32中，时钟树的配置如何影响外设和CPU的性能？

在STM32中，时钟树配置直接影响CPU和外设的性能：提高主频可以加速CPU的运算和指令执行速度，同时也会加快数据处理速度；而为不同外设选择合适的时钟频率，可以满足它们的性能需求，并在必要时降低功耗。

31. 如何在STM32上实现定时器中断？

在STM32上实现定时器中断，需要使用STM32CubeMX或库函数进行配置，主要步骤包括：**开启定时器时钟、初始化定时器参数（分频系数和重装载值）、使能定时器中断、并在中断回调函数中编写处理代码。**

32. 如何使用STM32的USART模块进行UART通信？

要使用STM32的USART模块进行UART通信，需要进行以下步骤：配置GPIO引脚、设置时钟源和波特率、并根据需要配置中断。首先，将USART的TX和RX引脚连接到外部设备，并配置这些GPIO端口为复用功能。其次，通过RCC寄存器使能USART时钟，并设置正确的波特率（Baud Rate）。最后，如果使用中断方式，需要使能相应的UART中断（如接收中断）并通过中断服务函数来处理数据。

33. STM32中的RTC（实时时钟）如何工作？

STM32的RTC（实时时钟）是一个在主电源断电后仍能继续工作的硬件计时器，它通过低功耗模式和电池（通常是锂电池）供电来维持时间和日历功能。其工作原理是：一个由外部时钟（通常是32.768kHz的LSE晶体）驱动的分频器，将时钟信号转换为一个秒级的计数器时钟，然后通过一个32位计数器进行累加计数。

34. 在STM32中如何实现低功耗模式？

在STM32中实现低功耗模式，主要通过选择不同的低功耗模式（如睡眠、停止、待机）并配置相应的系统时钟和外设，或者在运行模式下降低时钟频率并关闭不使用的设备。

35. 什么是实时操作系统（RTOS）？与普通操作系统有什么区别？

实时操作系统（RTOS）是一种专为需要快速、可预测响应的系统设计的操作系统，其核心特征是**实时性**，确保任务能在严格的时间限制内完成。**实时性**：这是RTOS最核心的特征，系统能对外部事件或数据在规定时间内做出及时且可预测的响应。**确定性**：RTOS的调度器和中断响应机制确保高优先级任务在任何情况下都能被及时执行，无论系统负载有多高。

36. RTOS中的任务调度机制是什么？有哪些常见的调度策略？

RTOS的任务调度机制是内核通过调度器，按预设策略分配CPU资源的过程

1. **抢占式优先级调度**：静态优先级（FreeRTOS默认）保证高优先级任务优先执行，动态优先级（继承/天花板）解决优先级反转；
2. **时间片轮转**：同优先级任务轮流执行，保证公平性；

37. 如何在FreeRTOS中创建一个任务？

在 FreeRTOS 中创建任务主要使用 `xTaskCreate()` 函数，该函数需要你提供任务函数、任务名称、堆栈大小、传递给任务的参数、优先级以及一个可选的任务句柄指针。创建任务后，需要调用 `vTaskStartScheduler()` 启动调度器，才能使任务开始运行

38. 在RTOS中如何实现任务间的通信？

在RTOS（实时操作系统）中，实现任务间通信的方法主要有**消息队列**、**信号量**、

39. 如何解决RTOS中的任务优先级反转问题？

任务优先级反转是指高优先级任务因等待低优先级任务释放的共享资源而被阻塞的现象。它会导致系统性能下降甚至死锁。解决方法主要有**优先级继承**（通过暂时提高低优先级任务的优先级来让其快速释放资源）和**优先级天花板协议**（为资源设定一个固定的优先级，所有获取资源的任务都将优先级提升到这个级别）

40. RTOS中的中断延迟如何影响系统性能？

“从外部事件置位中断请求，到 CPU 开始执行用户中断服务程序第一条指令之间的时间

1. **核心影响**：延迟过大会导致硬实时场景失效（安全事故）、软实时场景体验下降（数据丢失）、系统吞吐量降低；
2. **关键成因**：内核关中断时间、高优先级中断抢占、上下文切换是主要贡献因素；
3. **优化方向**：缩短临界区、合理配置中断优先级、ISR 轻量化、硬件加速，最终将延迟控制在场景允许的阈值内。

41. 在RTOS中如何实现信号量与互斥量？

在RTOS中，信号量和互斥量通常通过操作系统提供的API来实现，它们是用于任务间同步和保护共享资源的机制。信号量用于控制对数量有限的资源的访问，而互斥量则是一种特殊的二值信号量，用于确保对临界资源的独占访问。

42. RTOS中任务堆栈的大小如何配置和管理？

在RTOS中，任务堆栈大小通常是在创建任务时静态分配的，需要手动指定一个值。配置管理方法包括：

- **静态分配**：在创建任务前，在代码中定义一个固定大小的数组作为任务堆栈，并在创建任务时，将堆栈数组的首尾指针传递给创建任务的函数。
- **调整堆栈大小**：根据任务的复杂性和栈的消耗情况，手动调整堆栈大小的定义值。例如，在示例中，`TASK_CHECK_STK_SIZE` 被定义为 128，具体大小取决于代码的实际需求。

为了优化堆栈使用，可以：

- **估算**：通过分析任务代码，估算其运行过程中所需的最大堆栈空间。
- **监测**：在运行时，使用RTOS提供的堆栈检查功能（例如 [OS_TASK_OPT_STK_CHK](#)）来监测堆栈使用情况，并在需要时调整堆栈大小

43. 什么是RTOS中的时间片轮转调度？

时间片轮转调度算法：**将处理器时间分成固定长度的时间片，每个任务在一个时间片内执行。当时间片结束时，调度器会切换到下一个任务。**这种调度算法适用于多个任务具有相同优先级的情况，能够保证每个任务都有机会执行。

44. 在RTOS中如何处理多个任务的同步问题？

RTOS 中多任务同步的核心目标是**协调任务执行顺序、避免资源竞争、保证数据一致性**，需通过内核提供的同步机制实现。RTOS 中多任务同步需根据场景选择合适机制：

1. **二进制信号量**：任务 / 中断间简单同步；
2. **互斥量**：共享资源保护（解决优先级反转）；
3. **事件组**：多条件同步；
4. **计数信号量**：资源计数或批量事件；
5. **消息队列**：同步 + 数据传输；
6. **任务通知**：一对一轻量级同步。

45. 指针数组与数组指针的定义及区别，分别如何使用？

数组指针是指向**数组**的**指针**变量，而**指针数组**是一个**数组**，其中的元素都是**指针**类型。**数组指针**可以通过**指针**访问**数组**中的元素，而**指针数组**可以存储多个**指针**，每个**指针**指向不同的数据。

46. `const` 修饰指针时，“指针常量”与“常量指针”的本质差异是什么？

`const` 修饰指针的核心差异在于**约束对象不同**：

1. **常量指针**（`const 类型* p`）：约束指针指向的内容不可修改，指针本身可换指向；
2. **指针常量**（`类型* const p`）：约束指针本身的指向不可修改，指向的内容可修改；

47. 嵌入式中避免野指针方法

野指针（悬挂指针 / 无效指针）是指向已释放内存、非法地址或未初始化地址的指针。指针初始化赋 `NULL` / 合法地址，动态分配后检查返回值，释放后置 `NULL`；

48. 全局变量、静态全局变量、局部变量的存储区域及生命周期差异。

三类变量的核心差异在于**存储位置和生命周期**：

1. **全局变量**：`.data/.bss` 段，未初始化在 `.bss` 中（ram），初始化在 `.data` 中（flash）程序全程存在，多文件可见；
2. **静态全局变量**：存储同全局变量，但作用域限于本文件；
3. **普通局部变量**：栈中，函数调用期间存在，作用域局部；
4. **static 局部变量**：存储同全局变量，生命周期全程，但作用域局部。

49. 内存泄漏的原因，以及怎么避免

内存泄漏是指程序中动态分配的内存未被释放，导致内存资源被持续占用，最终耗尽系统内存。首先动态分配内容用完后及时释放，其次尽量使用静态内存

50. STM32 中，GPIO 引脚配置为输入模式时，上拉 / 下拉电阻的作用是什么？

1. **提供确定的默认电平**，避免引脚悬空导致的电平不确定；
2. **匹配外部电路**（如 I2C 总线），保护引脚硬件。

51. I2C 多主仲裁

I2C 总线支持多主设备（Multi-Master）挂载，当多个主设备同时发起通信时，需通过**仲裁机制**避免总线冲突，确保数据传输的唯一性。I2C 多主仲裁的核心是**线与逻辑 + 逐位电平对比**：

1. 多个主设备同时发起通信时，通过对比自身输出与总线电平，低电平优先；
2. 仲裁失败的设备放弃总线，切换为从设备，仲裁成功的设备继续传输；
3. 仲裁是无损的，且遵循地址 / 数据位的优先级规则。

52. 讲一讲 SPI 通信中，CPOL（时钟极性）和 CPHA（时钟相位）

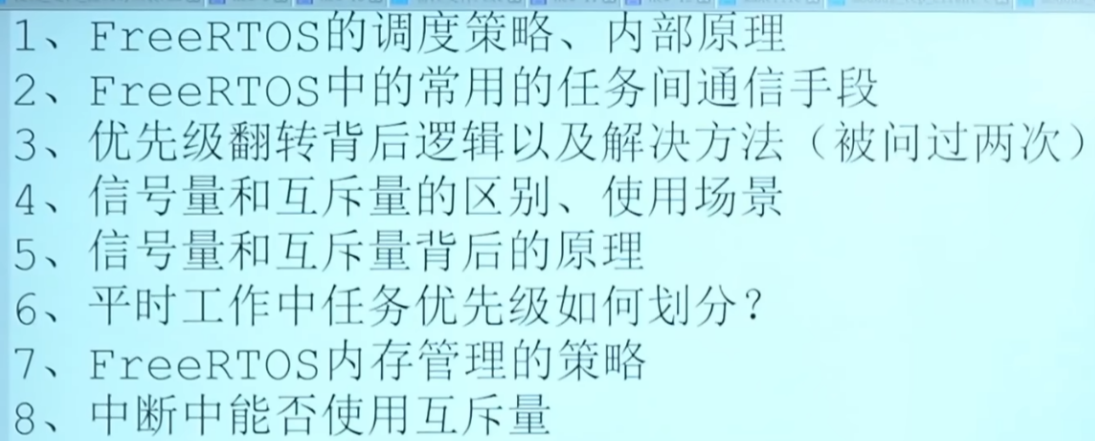
模式	CPOL	CPHA	空闲电平	采样边沿	数据更新边沿
0	0	0	低	上升沿（第 1 沿）	下降沿（第 2 沿）
1	0	1	低	下降沿（第 2 沿）	上升沿（第 1 沿）
2	1	0	高	下降沿（第 1 沿）	上升沿（第 2 沿）
3	1	1	高	上升沿（第 2 沿）	下降沿（第 1 沿）

53. 定时器用过吗，常见工作模式？

使用 STM32 的定时器（如 TIM1/TIM2/TIM3 等）实现定时中断、PWM 输出、输入捕获等功能。定时器的核心是通过**计数器**和**时钟源**的配合，实现对时间或外部事件的测量与控制。定时器是嵌入式系统的核心外设，我常用的工作模式包括：

1. **定时中断模式**：实现固定周期任务（如传感器采集）；

2. **PWM 输出模式**：控制电机转速、LED 亮度；
 3. **输入捕获模式**：测量外部信号频率 / 脉宽（如超声波测距）；
 4. **编码器模式**：电机测速与定位；
54. 外部中断的触发方式有哪些？中断优先级配置需遵循什么原则？
- 外部中断的触发方式包括**上升沿、下降沿、双边沿、电平触发**，需根据外部信号特性选择；中断优先级配置需遵循：
1. **按紧急度分配抢占优先级**，高紧急事件设高抢占优先级；
 2. **同抢占优先级按重要性分配响应优先级**；
55. 裸机系统与 RTOS 系统的核心差异？
- 裸机系统与 RTOS 系统的核心差异在于**任务管理和调度机制**：
1. 裸机是单任务 / 前后台架构，无抢占调度，依赖中断和轮询，适合简单、资源受限场景；
 2. RTOS 是多任务抢占式架构，内核调度器动态管理任务，支持同步互斥和实时保障，适合复杂、多任务、硬实时场景。
56. RTOS 中任务的状态有哪些？
- RTOS 中的任务通常有以下几种状态：**就绪状态、运行状态、阻塞状态和挂起状态**。
57. 任务间通信方式？用过哪些，什么场景？
58. 信号量的两种类型，分别适用于什么场景？
- 信号量主要分为二进制信号量和计数信号量：
1. **二进制信号量**：0/1 状态，适用于任务间 / 任务与中断的同步、单一共享资源的互斥访问；
 2. **计数信号量**：0~N 状态，适用于多实例资源池管理、事件累计触发（如批量数据处理）。
59. 消息队列的发送与接收过程，如何处理队列满 / 空的异常情况？
- 消息队列的发送过程是“检查队列状态→拷贝消息→唤醒接收任务”，接收过程是“检查队列状态→读取消息→唤醒发送任务”；队列满时可通过阻塞等待、覆写消息或丢弃新消息处理，队列空时可通过超时等待、永久阻塞或轮询重试处理。
60. 中断响应的过程
- 中断响应是**中央处理机在检测到中断请求后，中止当前程序执行并自动转入中断处理程序的过程**，由中断装置实现硬件层面的响应操作。该过程包含识别中断源、保存当前程序断点及标志状态、定位并执行中断服务程序等核心环节。
61. 中断服务函数与任务间通信的安全方式，为何不建议在 ISR 中直接调用复杂函数？
- ISR 与任务的安全通信方式包括二进制信号量（同步）、消息队列（数据）、事件标志组（多事件）及原子全局变量（简单标记），核心是“ISR 通知、任务处理”；不建议在 ISR 中调用复杂函数的原因是：中断需快进快出，复杂函数会导致实时性下降、资源竞争、栈溢出等问题，破坏系统稳定性。
62. 任务优先级的分配原则，高优先级任务长期占用 CPU 会导致什么问题？
63. RTOS 的内存池机制如何优化内存分配？
- RTOS 内存池通过**预分配固定大小的内存块 + 链表管理**，从根本上优化内存分配：消除碎片化、提升分配效率、保证实时性、简化越界检测；
64. 固件升级过程中突然断电，再次上电后如何保证系统能恢复并重新启动升级流程？
- 固件升级断电恢复的核心是“分区隔离 + 状态记录 + 校验机制”：通过独立的 Bootloader 区保证系统可启动，双固件区实现冗余备份，参数区记录升级状态和断点信息
65. 用过DMA吗？SPI 通信的 SD 卡读写在高速打印时频繁超时，如何通过 DMA 配置或缓存策略来平衡性能与稳定性？
- 66.

- 
- The image shows a Windows taskbar at the top with several open applications: '代码_文档_源码_列表_和...', 'new 9', 'new 10', '新文件.txt', 'new 11', 'new 12', 'newfile', 'modbus_tcp_client.c', and 'modbus_tcp...'. Below the taskbar, on a light blue background, is a numbered list of eight topics related to FreeRTOS.
- 1、FreeRTOS的调度策略、内部原理
 - 2、FreeRTOS中的常用的任务间通信手段
 - 3、优先级翻转背后逻辑以及解决方法（被问过两次）
 - 4、信号量和互斥量的区别、使用场景
 - 5、信号量和互斥量背后的原理
 - 6、平时工作中任务优先级如何划分？
 - 7、FreeRTOS内存管理的策略
 - 8、中断中能否使用互斥量